DS701 Homework 6: k-NN, the Curse and Naive Bayes

Released Tue Oct 13 · due Mon Oct 19, 11:59 pm on Gradescope

SOLUTIONS

Problem 1: k-NN by hand, and why scale matters

Practices: the k-NN prediction rule, the effect of feature units on Euclidean distance, and the bias–variance role of k.

Eight users of an app are described by x_1 = hours of use per week (0–10) and x_2 = fraction of sessions rated "satisfied" (0–1). Label A = renewed the subscription, B = cancelled.

point	P1	P2	P3	P4	P5	P6	P7	P8
x_1	4	8	6	0	2	7	10	1
x_2	0.2	0.9	0.7	0.2	0.0	0.1	1.0	0.5
label	A	A	A	A	B	B	B	B

A new user has $\mathbf{q} = (4,\ 0.1)$. Use Euclidean distance; since only the *ordering* of distances matters, you may work with squared distances throughout.

(a) Compute the squared distance from $\mathbf{q}$ to each of the eight points. Give the $k = 1$ and the $k = 3$ predictions (majority vote).

point	P1	P2	P3	P4	P5	P6	P7	P8
d^2	_____	_____	_____	_____	_____	_____	_____	_____

$k = 1$ prediction: _____ $k = 3$ prediction: _____

(b) A colleague points out that x_2 is a fraction while x_1 is in hours, and re-expresses x_2 as a score out of 10 (multiply the x_2 column and q_2 by 10). Recompute the squared distances, and give the new $k = 1$ and $k = 3$ predictions. Which prediction changed, and why exactly — which point(s) entered or left the neighbourhood? In one sentence, what does this say about running k-NN on raw features?

New $k = 1$ prediction: _____ new $k = 3$ prediction: _____

(c) What does $k = 8$ predict for *any* query in this dataset, and why? Using the vocabulary of Lecture 07 (bias/variance), explain what moving k from 1 toward 8 does to the decision boundary and to the two kinds of error, and say how k should be chosen in practice.

$k = 8$ predicts: _____

Solution

(a) $d^2(\mathbf{q}, P_i) = (x_1 - 4)^2 + (x_2 - 0.1)^2$: P1: $0 + 0.01 = 0.01$; P2: $16 + 0.64 = 16.64$; P3: $4 + 0.36 = 4.36$; P4: $16 + 0.01 = 16.01$; P5: $4 + 0.01 = 4.01$; P6: $9 + 0 = 9.00$; P7: $36 + 0.81 = 36.81$; P8: $9 + 0.16 = 9.16$. Sorted: P1 (A), P5 (B), P3 (A), P6 (B), P8 (B), P4 (A), P2 (A), P7 (B). $k = 1$: P1 $\Rightarrow$ **A**. $k = 3$: {P1, P5, P3} = A, B, A $\Rightarrow$ **A**.

(b) Now $x_2 \in \{2, 9, 7, 2, 0, 1, 10, 5\}$ and $q_2 = 1$; $d^2 = (x_1 - 4)^2 + (x_2 - 1)^2$: P1: $0 + 1 = 1$; P2: $16 + 64 = 80$; P3: $4 + 36 = 40$; P4: $16 + 1 = 17$; P5: $4 + 1 = 5$; P6: $9 + 0 = 9$; P7: $36 + 81 = 117$; P8: $9 + 16 = 25$. Sorted: P1 (A), P5 (B), P6 (B), P4 (A), P8 (B), P3 (A), P2 (A), P7 (B). $k = 1$: still P1 $\Rightarrow$ **A**. $k = 3$: {P1, P5, P6} = A, B, B $\Rightarrow$ **B** — flipped. Before rescaling, x_2 differences were at most 1 against x_1 differences of up to 10, so x_2 was effectively ignored and P3 ($x_1 = 6$, but a very different satisfaction, 0.7 vs 0.1) counted as close. After rescaling, P3's satisfaction gap costs 36 and it drops to sixth; P6 (similar satisfaction, $x_1 = 7$) takes its place. k-NN's answer depends on the *units* you happened to record — so scale (standardize) features first; otherwise the largest-range feature silently decides. *Common mistake:* comparing distances instead of squared distances is fine, but mixing the two within one ranking is not.

(c) $k = 8$ uses every point, so it always returns the majority class of the whole training set — here a 4–4 tie (any tie-break rule; the point is that the query's location no longer matters). Small k: jagged boundary that follows individual points, low bias, high variance (a different training sample moves the boundary a lot). Large k: smooth boundary, low variance, high bias — at $k = N$ it ignores $\mathbf{x}$ entirely. Choose k on held-out data (cross-validation over a grid of k), never by training error, which is trivially perfect at $k = 1$.

Problem 2: The curse of dimensionality, in numbers

Practices: quantifying "nearest is meaningless" with two short calculations.

Suppose N points are spread uniformly in the unit hypercube $[0, 1]^d$. To find the k nearest neighbours of a query, k-NN needs a small box around the query that contains a fraction $f = k/N$ of the data.

(a) Show that a hypercube of side length s inside $[0, 1]^d$ captures a fraction s^d of the data, so the side length needed to capture fraction f is $s = f^{1/d}$. (Two or three lines.)

(b) For $f = 0.1$ compute s for $d = 1, 2, 10, 100$. You may use $0.1^{0.5} \approx 0.316$, $0.1^{0.1} \approx 0.794$, $0.1^{0.01} \approx 0.977$. In one or two sentences, what does the $d = 100$ answer mean for the word "neighbourhood" in k-NN?

d	1	2	10	100
s	_____	_____	_____	_____

(c) Lecture 09 showed the fraction of a unit ball's volume within ϵ of its surface is $1-(1-\epsilon)^d$. For $\epsilon = 0.1$ compute it at $d = 2, 10, 50$ (use $0.9^{10} \approx 0.349$, $0.9^{50} \approx 0.005$). Combine this with the fact that $d_{\max}/d_{\min} \to 1$: why does "the nearest neighbour" stop carrying information as d grows?

Fraction within $\epsilon = 0.1$ of the surface:

$d = 2$: _____ $d = 10$: _____ $d = 50$: _____

(d) A teammate proposes to fix a poorly performing k-NN model on 500 features by "just collecting more data". Using a 10-bins-per-dimension argument (with 10^6 points, how many points per bin at $d = 3$? at $d = 6$? at $d = 12$?), say why that plan fails, and name the two remedies the lecture actually recommends.

Points per bin — $d = 3$: _____ $d = 6$: _____ $d = 12$: _____

Solution

(a) A box of side s has volume s^d; the unit cube has volume 1. Uniform points land in a region with probability equal to its volume, so the expected fraction of points in the box is $s^d/1 = s^d$. Setting $s^d = f$ gives $s = f^{1/d}$.

(b) $d = 1$: 0.1; $d = 2$: 0.316; $d = 10$: 0.794; $d = 100$: 0.977. In 100 dimensions, a box holding just 10% of the data must span 97.7% of the range of *every* coordinate — it is essentially the whole cube. The "neighbourhood" is not local: the k neighbours are typical points, not points that resemble the query.

(c) $d = 2$: $1 - 0.81 = 0.19$; $d = 10$: $1 - 0.349 = 0.651$; $d = 50$: $1 - 0.005 = 0.995$. So in 50-D, 99.5% of everything "within distance r" of the query sits in the outer 10% shell — at distance $\approx r$. Every candidate neighbour is about equally far, which is exactly $d_{\max}/d_{\min} \to 1$: the closest point is barely closer than the farthest, so which point is "nearest" is decided by noise and its label tells you little about the query.

(d) 10^6 points, 10^d bins: $d = 3$: $10^6/10^3 = 1000$ points per bin; $d = 6$: 1 point per bin; $d = 12$: 10^{-6}, i.e. one point per million bins. Keeping the density fixed as d grows needs 10^d points — exponential in d; at $d = 500$ no dataset is remotely large enough, so "more data" cannot make the space dense. Remedies from the lecture: use a different metric less affected by concentration (e.g. cosine distance), or reduce the dimension first (feature selection / dimensionality reduction such as PCA, coming in Lecture 13).

Problem 3: Naive Bayes on symptoms, and the SVM margin as geometry

Practices: Bayes-rule classification by counting, Laplace smoothing, and the distance-to-a-hyperplane computation behind support vector machines.

Part I — Naive Bayes. A clinic recorded three symptoms and a lab-confirmed diagnosis for ten patients:

#	Fever	Cough	Headache	Flu
1	Y	Y	Y	Y
2	Y	Y	Y	Y
3	Y	Y	Y	Y

#	Fever	Cough	Headache	Flu
4	Y	Y	N	Y
5	Y	Y	Y	N
6	N	Y	Y	N
7	N	Y	N	N
8	N	N	N	N
9	N	N	N	N
10	N	N	N	N

A new patient presents with **Fever = Y, Cough = N, Headache = Y**.

(a) Estimate the class priors and the six conditional probabilities Naive Bayes needs for this patient, P(symptom value | Flu) and P(symptom value | no Flu), by counting. Compute the Naive Bayes score $P(C)\prod_j P(x_j \mid C)$ for each class and give the prediction. What single number decided it?

class C	$P(C)$	$P(F{=}Y \mid C)$	$P(C{=}N \mid C)$	$P(H{=}Y \mid C)$	score
Flu	_____	_____	_____	_____	_____
no Flu	_____	_____	_____	_____	_____

Prediction: _____

(b) Redo the scores with Laplace smoothing $\alpha = 1$: each conditional becomes $\dfrac{\#\{x_j = v \text{ and class } C\} + \alpha}{\#\{\text{class } C\} + \alpha V_j}$, where $V_j = 2$ is the number of values symptom j can take (leave the priors unsmoothed). Give the smoothed prediction and the smoothed posterior $P(\text{Flu} \mid \mathbf{x})$ = (Flu score) / (sum of both scores). Did smoothing change the prediction, the confidence, or both? Round to 3 decimals.

Smoothed prediction: _____ $P(\text{Flu} \mid \mathbf{x})$: _____

(c) Suppose the clinic added a fourth column, "Chills", that happens to be identical to Fever in every row. Say what Naive Bayes would do to the fever evidence, why that violates the "naive" assumption, and — in one or two sentences — why the classifier often still picks the right class despite such violations.

Solution

(a) Priors: $P(\text{Flu}) = 4/10 = 0.4$, $P(\text{no}) = 6/10 = 0.6$. Flu rows 1–4: $P(F{=}Y \mid \text{Flu}) = 4/4 = 1$, $P(C{=}N \mid \text{Flu}) = 0/4 = 0$, $P(H{=}Y \mid \text{Flu}) = 3/4$. No-flu rows 5–10: $P(F{=}Y \mid \text{no}) = 1/6$, $P(C{=}N \mid \text{no}) = 3/6 = 1/2$, $P(H{=}Y \mid \text{no}) = 2/6 = 1/3$. Scores: Flu: $0.4 \cdot 1 \cdot 0 \cdot 0.75 = 0$; no: $0.6 \cdot \frac{1}{6} \cdot \frac{1}{2} \cdot \frac{1}{3} = 0.0167$. Prediction: **no flu** — decided entirely by the single zero: no training flu patient lacked a cough, so "no cough" vetoes flu regardless of fever and headache.

(b) Smoothed conditionals (+1 over +2): Flu: $P(F{=}Y) = 5/6 = 0.833$, $P(C{=}N) = 1/6 = 0.167$, $P(H{=}Y) = 4/6 = 0.667$; no: $P(F{=}Y) = 2/8 = 0.25$, $P(C{=}N) = 4/8 = 0.5$, $P(H{=}Y) = 3/8 = 0.375$. Scores: Flu: $0.4 \cdot 0.833 \cdot 0.167 \cdot 0.667 = 0.037$; no: $0.6 \cdot 0.25 \cdot 0.5 \cdot 0.375 = 0.028$. Prediction flips to **Flu**; posterior $P(\text{Flu} \mid \mathbf{x}) = 0.037/(0.037{+}0.028) = 0.568$. Both changed: the veto is gone (an unseen combination is now merely unlikely, not impossible), and the fever evidence — 0.833 vs 0.25, a factor above 3 — is allowed to weigh in. Note the modest posterior: with four flu rows the estimates are crude, and smoothing says so.

(c) Naive Bayes multiplies in $P(\text{Chills} \mid C)$ as an independent factor, which equals $P(\text{Fever} \mid C)$ — it counts the fever evidence *twice* (the fever likelihood ratio gets squared). That is exactly what conditional independence forbids: given the class, knowing Fever tells you Chills with certainty. It often does not matter because classification only needs the *arg max* to be right; double-counting distorts the probabilities (over-confidence) but tends to push the score further in the direction the evidence already pointed, so the winner is usually unchanged.